Machine Learning

Support Vector Machines

Sebastian Engelke

Master in Business Analytics

UNIVERSITÉ
DE GENÈVE

Support vector machines

- `Support vector machines` (SVM) enlarge the feature space in an automatic way.

Support vector machines

- Support vector machines (SVM) enlarge the feature space in an automatic way.
- A deep mathematical result is that the optimal support vector classifier (the solution to the optimization problem) can be written as the linear function

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i \langle x, x_i \rangle,$$

  where there is one parameter α_i per training observation, and the inner product is defined as $\langle x, z \rangle = \sum_{j=1}^{p} x_j z_j$.
- In fact, all α_i are zero exept those for the support vectors.
- You can see an inner product as a similarity measure; here it is the correlation!
- The SVM simply replaces the usual inner product by a more general similarity measure, called a kernel.

The kernel trick

- The SVM classifies the classes according to the, in general, non-linear function

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i K(x, x_i),$$

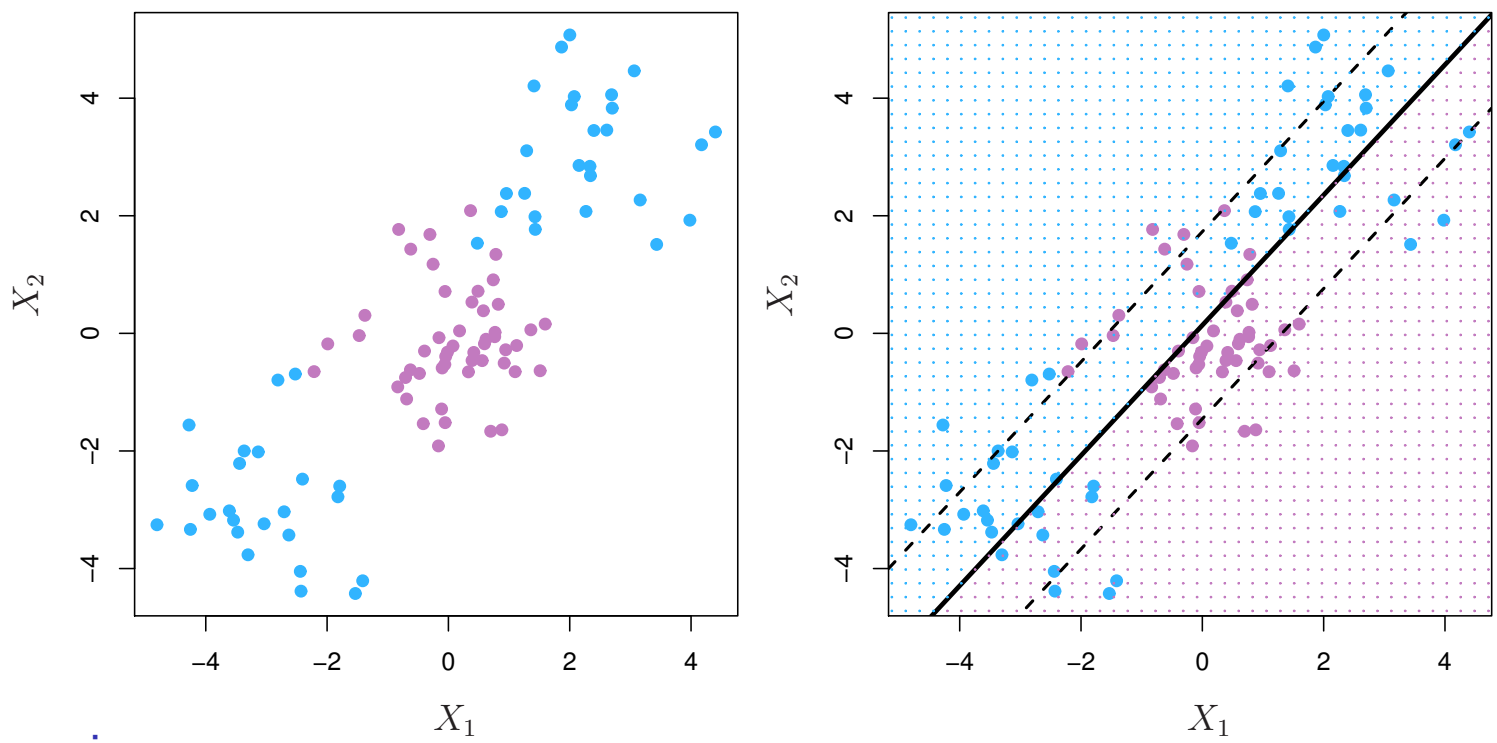
  where $K(x, z)$ is a (positive definite) kernel that measure similarity between $x, z \in \mathbb{R}^p$.
- By doing this, we implicitly enlarge the feature space, without actually doing it explicitly! This is called the kernel trick.

The kernel trick

- The SVM classifies the classes according to the, in general, non-linear function

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i K(x, x_i),$$

  where $K(x, z)$ is a (positive definite) kernel that measure similarity between $x, z \in \mathbb{R}^p$.
- By doing this, we implicitly enlarge the feature space, without actually doing it explicitly! This is called the kernel trick.
- With the linear kernel $K(x, z) = \langle x, z \rangle$ we recover the SV classifier.



The kernel trick: examples

The polynomial kernel

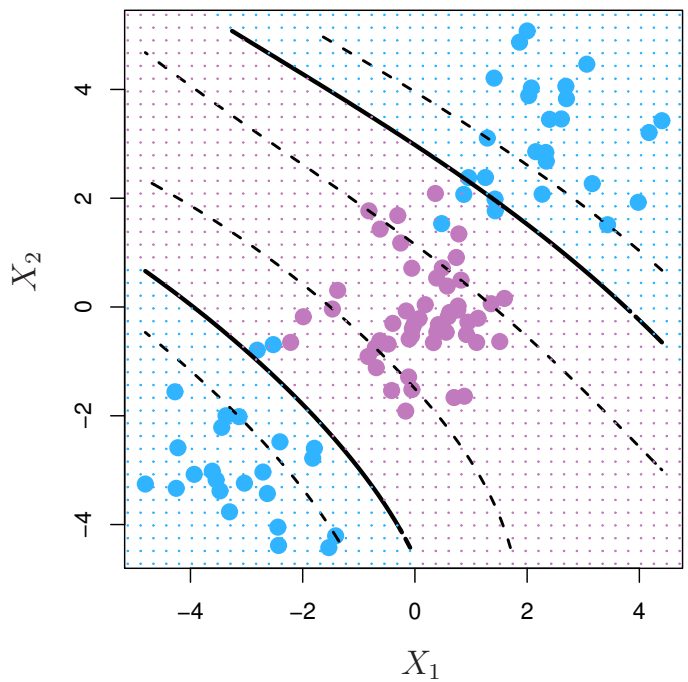
$$K(x,z) = \left(1 + \gamma \sum_{j=1}^{p} x_j z_j\right)^d$$

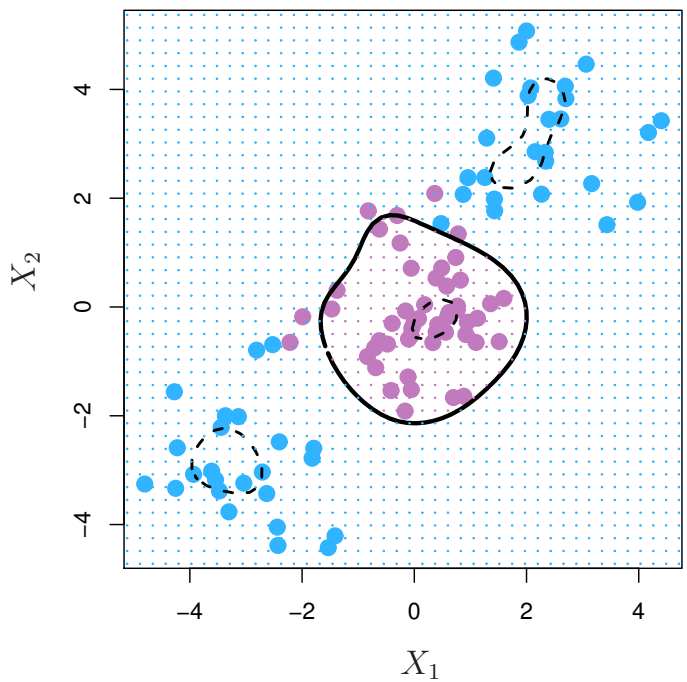
- Corresponds to fitting a SV classifier in a higher dimensional space with all polynomials of degree d!
- The tuning parameter γ controls the scale.

The radial/Gauss kernel

$$K(x,z) = \exp\left(-\gamma \sum_{j=1}^{p} (x_j - z_j)^2\right)$$

- Corresponds to fitting a SV classifier in a infinite dimensional space!
- The tuning parameter γ controls how "local" the fit is.





Support vector machines: comments

- SVMs are a very `powerful and flexible tool` for classification.
- The mathematics are quite hard, but thanks to the `kernel trick` we do not need to know the details!
- For fitting, we `only compute` all similarities $K(x_i, x_j)$ for all $\binom{n}{2}$ distinct pairs $i, j = 1, \ldots, n$ in the training set.

Support vector machines: comments

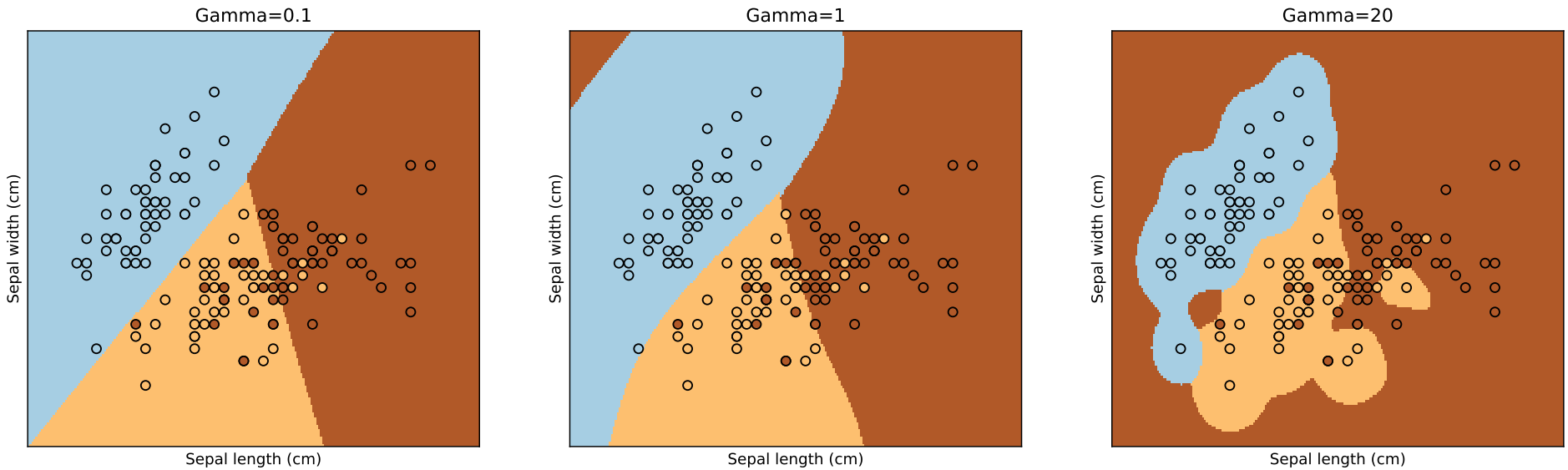
- SVMs are a very powerful and flexible tool for classification.
- The mathematics are quite hard, but thanks to the kernel trick we do not need to know the details!
- For fitting, we only compute all similarities $K(x_i, x_j)$ for all $\binom{n}{2}$ distinct pairs $i, j = 1, \ldots, n$ in the training set.
- Many different kernels have been develloped, but even the "standard" ones perform usually well.

Support vector machines: comments

- SVMs are a very powerful and flexible tool for classification.
- The mathematics are quite hard, but thanks to the kernel trick we do not need to know the details!
- For fitting, we only compute all similarities $K(x_i, x_j)$ for all $\binom{n}{2}$ distinct pairs $i, j = 1, \ldots, n$ in the training set.
- Many different kernels have been develloped, but even the "standard" ones perform usually well.
- For more than two classes, there are simple extensions, so-called One-Versus-One classification, or One-Versus-All classification.
- For more details see Chapter 9 in [ISL].

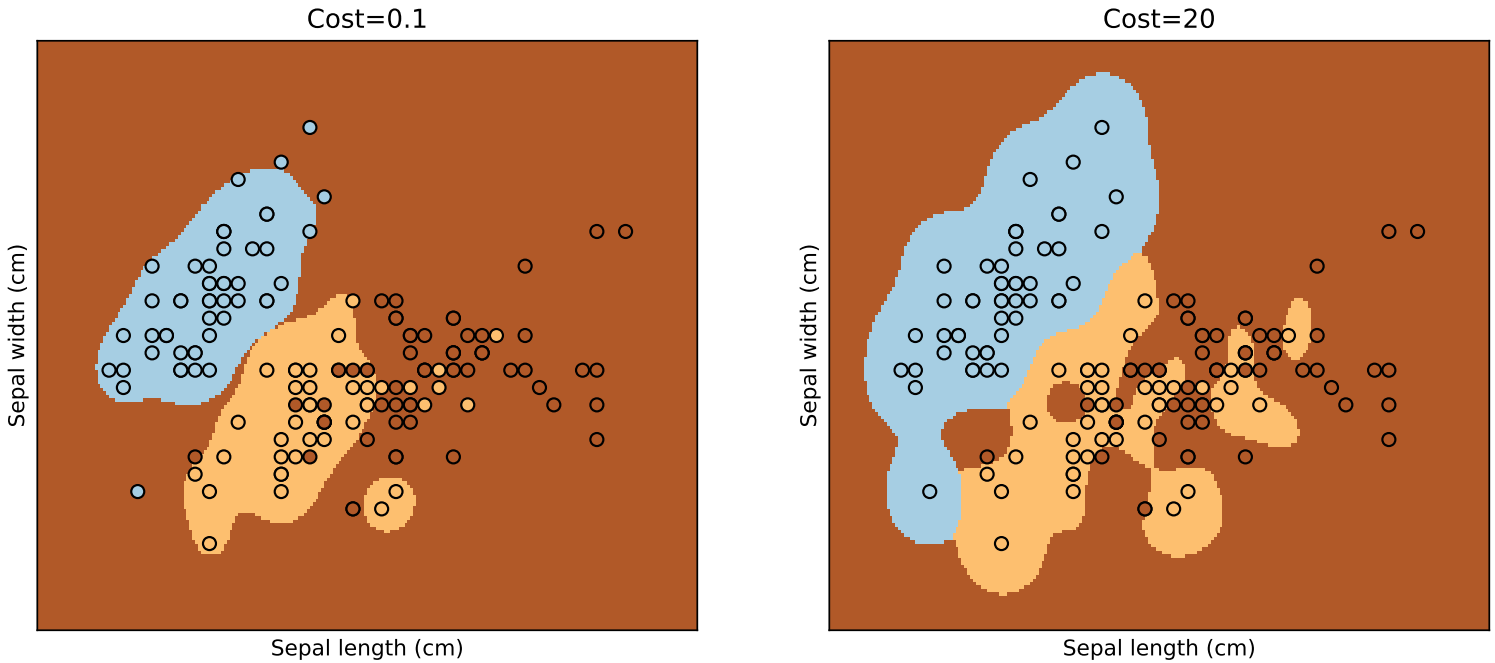
Support vector machines in python

```
from sklearn.svm import SVC
iris = sklearn.datasets.load_iris()
X, y = iris.data[:, :2], iris.target  # we only keep the first two features: 'Sepal length', 'Sepal width'
fig, axs = plt.subplots(1,3,figsize=(6*3,5))
gammas = [0.1,1,20]
for i, gamma in enumerate(gammas):
    svm = SVC(C=1, kernel='rbf', gamma=gamma)
    svm.fit(X, y)
    x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
    y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
    h = .02  # step size in the mesh
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    axs[i].pcolormesh(xx, yy, Z, cmap=plt.cm.Paired)
    axs[i].scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.Paired)
plt.show()
```



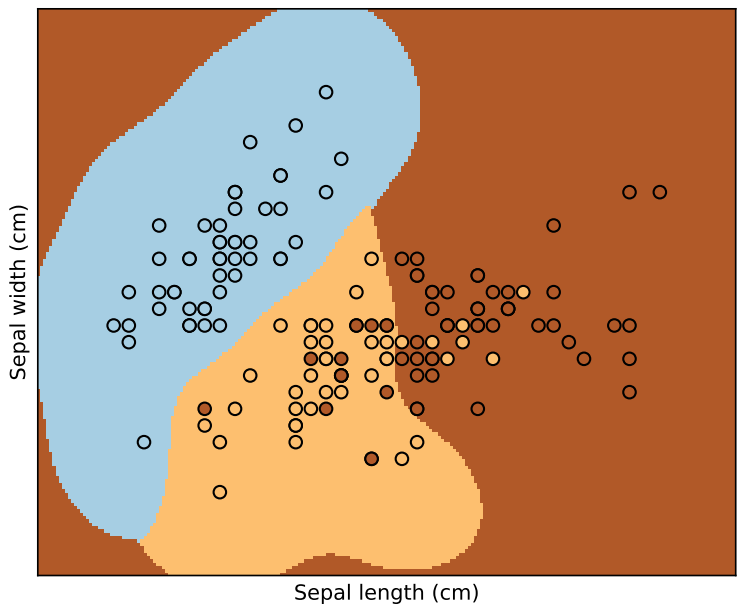
Support vector machines in python

```
from sklearn.svm import SVC
iris = sklearn.datasets.load_iris()
X, y = iris.data[:, :2], iris.target  # we only keep the first two features: 'Sepal length', 'Sepal width'
fig, axs = plt.subplots(1,3,figsize=(6*3,5))
costs = [0.1,20]
for i, cost in enumerate(costs):
    svm = SVC(C=cost, kernel='rbf', gamma=20)
    svm.fit(X, y)
    x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
    y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
    h = .02  # step size in the mesh
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    axs[i].pcolormesh(xx, yy, Z, cmap=plt.cm.Paired)
    axs[i].scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.Paired)
plt.show()
```

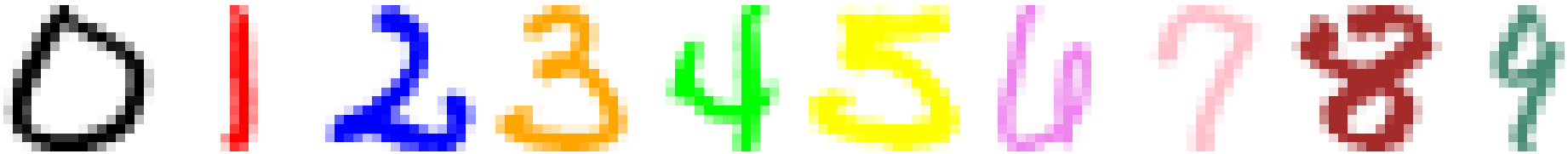


Support vector machines in python

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, KFold
iris = sklearn.datasets.load_iris()
X, y = iris.data[:, :2], iris.target  # we only keep the first two features: 'Sepal length', 'Sepal width'
hyper_parameters = {"C":[0.5,0.8,1,1.2,1.5], "gamma":[0.1,1,3,5,7,10,15]}
folds = KFold(n_splits=5, shuffle=True, random_state=1)
svmCV = GridSearchCV(estimator=SVC(kernel='rbf'), param_grid=hyper_parameters, cv=folds)
svmCV.fit(X, y)
print(svmCV.best_params_)# Out: {'C': 1.2, 'gamma': 5}
x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
h = .02  # step size in the mesh
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
Z = svmCV.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Paired);
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.Paired);
```



Digit classification with SVMs



- We can update our results on the digit classification.
- We use SVMs (One-Versus-One) with linear, polynomial and radial kernels.
- The tuning parameters the value of C were found by 10-fold-CV.

- linreg: direct linear regression;
- LDA1: LDA based on the values x_i;
- LDA2: LDA based on x_i and x_i^2;
- QDA: different cov. matrix for each class;
- log: logistic regression (multinomial).
- log-ridge: logistic regression with ridge.
- log-lasso: logistic regression with lasso.
- RF: random forest.
- linear SVM: linear support vector classifier.
- radial SVM: SVM with radial kernel.

	Training error	Test error
linreg	7.6%	13.1%
LDA1	6.2%	11.5%
LDA2	3.9%	10.2%
QDA	1.8%	13.5%
log	0.01%	11.1%
log-ridge	4.1%	9.0%
log-lasso	2.7%	8.8%
RF	0%	5.9%
linear SVM	1.5%	6.5%
radial SVM	0.6%	5.4%